

Захищений доступ

ssh gpg

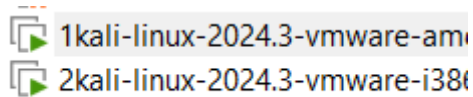
Кувавiна Софiя

Хiд роботи

Захищений доступ з використанням SSH

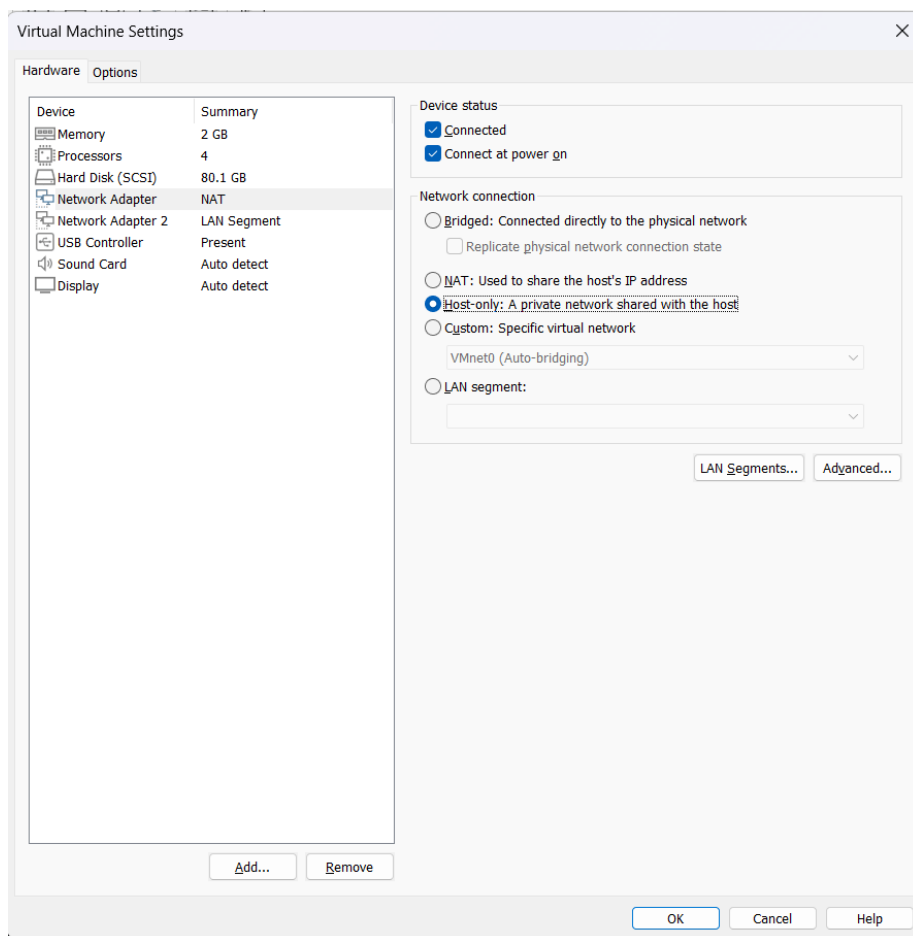
Запустiть двi виртуальнi машини з Linux, пiдключiть їх в загальну виртуальну мережу.

Я створила 2 виртуальнi машини калi лiнукс



Пiдключаю машини до спiльної мережi

У вкладцi Network Adapter для обох машин обираю Host-only



1.1. Задайте відповідні мережні адреси і утилітою ping перевірте досяжність між ними. Нехай VM Client має IP адреса 192.168.74.10, а VM Server - 192.168.74.100.

Вводимо для обох машин:

```
(kali@kali)-[~]  
$ sudo nano /etc/network/interfaces
```

Для VM Client:

```
GNU nano 8.1  
# This file describes the network interfaces available on your system  
# and how to activate them. For more information, see interfaces(5).  
  
source /etc/network/interfaces.d/*  
  
# The loopback network interface  
auto lo  
iface lo inet loopback  
auto eth0  
iface eth0 inet static  
    address 192.168.74.10  
    netmask 255.255.255.0  
    gateway 192.168.74.1
```

Для VM Server:

```
GNU nano 8.1  
# This file describes the network interfaces available on your system  
# and how to activate them. For more information, see interfaces(5).  
  
source /etc/network/interfaces.d/*  
  
# The loopback network interface  
auto lo  
iface lo inet loopback  
auto eth0  
iface eth0 inet static  
    address 192.168.74.100  
    netmask 255.255.255.0  
    gateway 192.168.74.1
```

Перезавантажую систему

```
(kali@kali)-[~]  
$ sudo systemctl restart networking
```

Перевірка з'єднання між машинами

На VM Client ввожу:

Так як з'являються відповіді – все правильно.

```
(kali㉿kali)-[~]  
$ ping 192.168.74.100  
  
PING 192.168.74.100 (192.168.74.100) 56(84) bytes of data.  
64 bytes from 192.168.74.100: icmp_seq=1 ttl=64 time=2.59 ms  
64 bytes from 192.168.74.100: icmp_seq=2 ttl=64 time=2.23 ms  
64 bytes from 192.168.74.100: icmp_seq=3 ttl=64 time=0.409 ms  
64 bytes from 192.168.74.100: icmp_seq=4 ttl=64 time=0.729 ms  
64 bytes from 192.168.74.100: icmp_seq=5 ttl=64 time=0.449 ms  
64 bytes from 192.168.74.100: icmp_seq=6 ttl=64 time=1.33 ms  
64 bytes from 192.168.74.100: icmp_seq=7 ttl=64 time=0.687 ms  
64 bytes from 192.168.74.100: icmp_seq=8 ttl=64 time=1.05 ms  
64 bytes from 192.168.74.100: icmp_seq=9 ttl=64 time=1.28 ms  
64 bytes from 192.168.74.100: icmp_seq=10 ttl=64 time=0.713 ms  
64 bytes from 192.168.74.100: icmp_seq=11 ttl=64 time=0.382 ms  
64 bytes from 192.168.74.100: icmp_seq=12 ttl=64 time=0.408 ms  
64 bytes from 192.168.74.100: icmp_seq=13 ttl=64 time=0.381 ms  
64 bytes from 192.168.74.100: icmp_seq=14 ttl=64 time=0.414 ms  
64 bytes from 192.168.74.100: icmp_seq=15 ttl=64 time=2.09 ms  
64 bytes from 192.168.74.100: icmp_seq=16 ttl=64 time=1.77 ms  
64 bytes from 192.168.74.100: icmp_seq=17 ttl=64 time=1.18 ms  
64 bytes from 192.168.74.100: icmp_seq=18 ttl=64 time=1.05 ms  
64 bytes from 192.168.74.100: icmp_seq=19 ttl=64 time=0.417 ms  
64 bytes from 192.168.74.100: icmp_seq=20 ttl=64 time=0.536 ms  
64 bytes from 192.168.74.100: icmp_seq=21 ttl=64 time=2.08 ms  
64 bytes from 192.168.74.100: icmp_seq=22 ttl=64 time=0.563 ms  
64 bytes from 192.168.74.100: icmp_seq=23 ttl=64 time=0.642 ms  
64 bytes from 192.168.74.100: icmp_seq=24 ttl=64 time=0.568 ms  
64 bytes from 192.168.74.100: icmp_seq=25 ttl=64 time=1.00 ms  
64 bytes from 192.168.74.100: icmp_seq=26 ttl=64 time=0.330 ms  
64 bytes from 192.168.74.100: icmp_seq=27 ttl=64 time=1.57 ms  
64 bytes from 192.168.74.100: icmp_seq=28 ttl=64 time=0.970 ms  
64 bytes from 192.168.74.100: icmp_seq=29 ttl=64 time=0.753 ms  
64 bytes from 192.168.74.100: icmp_seq=30 ttl=64 time=1.12 ms  
64 bytes from 192.168.74.100: icmp_seq=31 ttl=64 time=0.578 ms  
64 bytes from 192.168.74.100: icmp_seq=32 ttl=64 time=2.63 ms  
64 bytes from 192.168.74.100: icmp_seq=33 ttl=64 time=0.561 ms  
64 bytes from 192.168.74.100: icmp_seq=34 ttl=64 time=37.7 ms  
64 bytes from 192.168.74.100: icmp_seq=35 ttl=64 time=0.357 ms  
64 bytes from 192.168.74.100: icmp_seq=36 ttl=64 time=1.77 ms  
64 bytes from 192.168.74.100: icmp_seq=37 ttl=64 time=0.496 ms
```

1.2. На VM Server перевірте, що сервіс ssh запущено:

```
#ps -A | grep ssh
```

```
1615 ? 00:00:00 sshd
```

```
2402 ? 00:00:00 sshd
```

Перевіряю наявність SSH-сервера і його статус, бачу, що він неактивний(inactive (dead))
тому треба запустити

```
(kali㉿kali)-[~]
$ ps -A | grep ssh

1373 ?          00:00:00 ssh-agent

(kali㉿kali)-[~]
$ sudo apt list --installed | grep openssh-server

WARNING: apt does not have a stable CLI interface. Use with caution in scripts.

openssh-server/now 1:9.7p1-7 i386 [installed,local]

(kali㉿kali)-[~]
$ sudo systemctl status ssh

o ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: disabled)
   Active: inactive (dead)
     Docs: man:sshd(8)
           man:sshd_config(5)
```

запускаю його і перевіряю - все працює.

```
(kali㉿kali)-[~]
$ sudo systemctl start ssh

(kali㉿kali)-[~]
$ sudo systemctl enable ssh
Synchronizing state of ssh.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable ssh
Created symlink '/etc/systemd/system/sshd.service' → '/usr/lib/systemd/system/ssh.service'.
Created symlink '/etc/systemd/system/multi-user.target.wants/ssh.service' → '/usr/lib/systemd/system/ssh.service'.

(kali㉿kali)-[~]
$ sudo netstat -tuln | grep :22

tcp        0      0 0.0.0.0:22          0.0.0.0:*           LISTEN
tcp6       0      0 :::22              :::*                LISTEN
```

1.3. 3 VM Client встановить захищене з'єднання з Server:

#ssh -l root 192.168.74.100

* Замість root можна використовувати будь-який активний обліковий запис.

Далі, підтвердіть автентичність відкритого ключа сервера і введіть пароль даного користувача.

Ви відкрили ssh сеанс з VM Server.

Виконайте кілька команд на віддаленому комп'ютері і закрийте з'єднання:

#exit

Спочатку ввожу ssh -l root 192.168.74.100

підтверджую автентичність ключа, ввівши yes, і ввожу пароль, далі виконую кілька команд і завершую сесію.

```

(kali㉿kali)-[~]
$ ssh -l root 192.168.74.100

root@192.168.74.100's password:
Linux kali 6.8.11-686-pae #1 SMP PREEMPT_DYNAMIC Kali 6.8.11-1kali2 (2024-05-30) i686

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
(kali㉿kali)-[~]
# ls

(kali㉿kali)-[~]
# whoami
root

(kali㉿kali)-[~]
# exit

Connection to 192.168.74.100 closed.

```

1.4. Запуск команди на VM Server:

#ssh -l <remoteuser> <remotehost> <command>

#ssh -l root 192.168.74.100 uptime

root@192.168.74.100's password:

10:37:57 up 2:25, 5 users, load average: 0.00, 0.00, 0.00

Щоб запустити віддалено інтерактивну команду (програму):

#ssh -t -l <remoteuser> <remotehost> vi

щоб перевірити тривалість роботи VM Server, ввожу:

```

(kali㉿kali)-[~]
$ ssh -l root 192.168.74.100 uptime

root@192.168.74.100's password:
14:23:13 up 1:31, 4 users, load average: 0.01, 0.06, 0.03

```

Щоб відкрити текстовий редактор vi на VM Server, виконую

```

(kali㉿kali)-[~]
$ ssh -t -l root 192.168.74.100 vi

root@192.168.74.100's password:
Connection to 192.168.74.100 closed.

```

Відкривається редактор vi



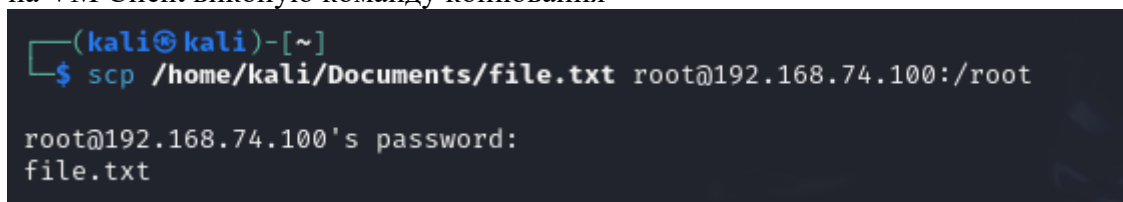
1.5 Копіювання файлів по захищеному з'єднанню:

#scp <file> <remoteuser>@<remotehost>:

#scp <file> 192.168.74.10:

* Якщо в команді scp не задано ім'я користувача, то використовується поточний обліковий запис і пароль.

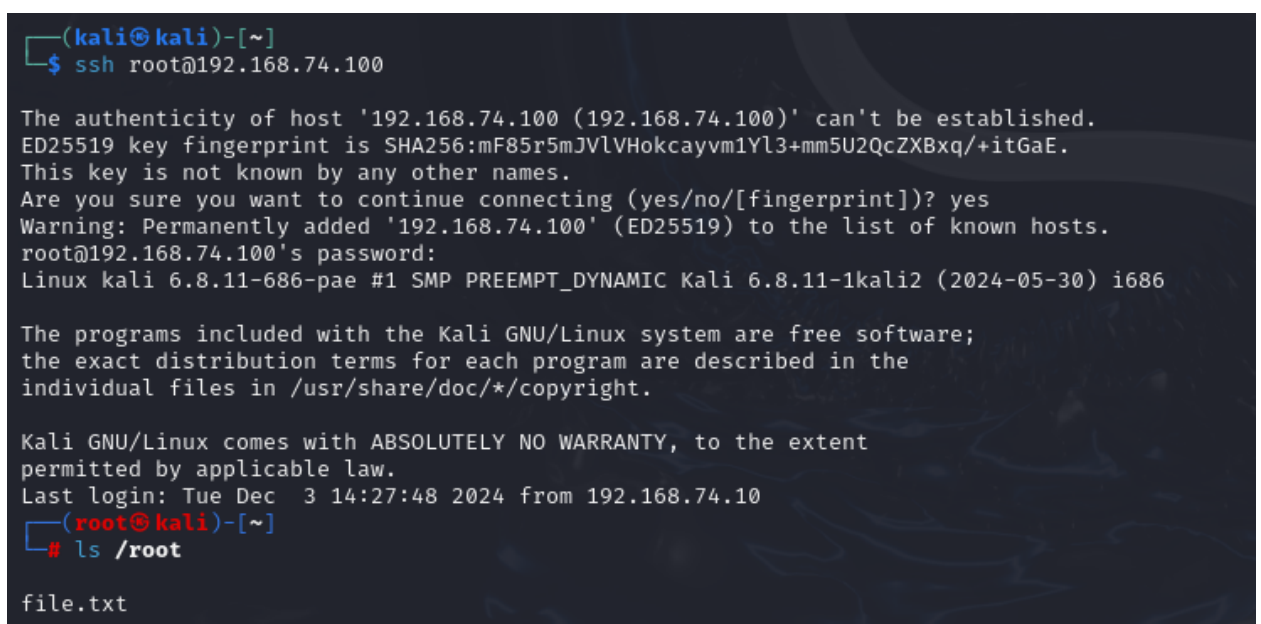
на VM Client виконую команду копіювання



І перевіряю на VM Server:

Вхожу в систему як root і ввожу команду ls

Бачимо, що все спрацювало



ЕРІТАРН

Застосування GPG для шифрування і цифрового підпису

2.1. Створіть пару відкритий / таємний ключ:

```
#gpg --gen-key
```

Введіть необхідні параметри суб'єкта і ключа.

Виберіть тип ключа, придатний як для шифрування, так і для цифрового підпису.

* У документації уточніть параметри команд, використовуваних в завданнях.

Щоб переглянути атрибути існуючих ключів, скористайтеся командами:

```
#gpg --list-secret-keys
```

```
#gpg --list-public-keys
```

```
/root/.gnupg/pubring.gpg
```

```
-----  
pub 1024D/A7DC03D8 2017-04-11 User1 (User1 Key) <user1@gmail.com>
```

```
sub 2048g/06918E9D 2017-04-11
```

Тут pub(public) - відкритий ключ, sec(secret:) - таємний ключ,

sub и ssb - вторинні ключі (в даному завданні не використовуються).

Кількість біт: 1024 або 2048;

Алгоритм шифрування: D(DSA);

key ID: A7DC03D8

Дата створення: 2017-04-11

user ID: User1 <user1@gmail.com>

Експорт відкритого ключа (збереження у файл):

```
#gpg --export -a "User1" >user1.pubkey
```

Скопіюйте цей ключ на VM Server:

```
#scp user1.pubkey 192.168.74.100:
```

Створіть текстовий файл та введіть в його декілька строк тексту:

```
#touch testfile
```

Створіть ЦП цього файла:

```
#gpg --clearsign testfile
```

Передайте файл з ЦП на VM Server:

```
#scp testfile.asc 192.168.74.100:
```

Я створюю пару ключів на VM Client, вводячи всі дані

```

(kali㉿kali)-[~]
$ gpg --gen-key

gpg (GnuPG) 2.2.43; Copyright (C) 2023 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Note: Use "gpg --full-generate-key" for a full featured key generation dialog.

GnuPG needs to construct a user ID to identify your key.

Real name: Sofiia
Email address: kuvavinasofia@gmail.com
You selected this USER-ID:
  "Sofiia <kuvavinasofia@gmail.com>"

Change (N)ame, (E)mail, or (O)kay/(Q)uit? o
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
gpg: /home/kali/.gnupg/trustdb.gpg: trustdb created
gpg: directory '/home/kali/.gnupg/openpgp-revocs.d' created
gpg: revocation certificate stored as '/home/kali/.gnupg/openpgp-revocs.d/CE47418DCA723B3D7FB85FA062D1D2171937222A.rev'
public and secret key created and signed.

pub   rsa3072 2024-12-03 [SC] [expires: 2027-12-03]
       CE47418DCA723B3D7FB85FA062D1D2171937222A
uid     Sofiia <kuvavinasofia@gmail.com>
sub   rsa3072 2024-12-03 [E] [expires: 2027-12-03]

```

І перевіряю створені ключі

```

(kali㉿kali)-[~]
$ gpg --list-public-keys

gpg: checking the trustdb
gpg: marginals needed: 3  completes needed: 1  trust model: pgp
gpg: depth: 0  valid: 1  signed: 0  trust: 0-, 0q, 0n, 0m, 0f, 1u
gpg: next trustdb check due at 2027-12-03
/home/kali/.gnupg/pubring.kbx

pub   rsa3072 2024-12-03 [SC] [expires: 2027-12-03]
       CE47418DCA723B3D7FB85FA062D1D2171937222A
uid     [ultimate] Sofiia <kuvavinasofia@gmail.com>
sub   rsa3072 2024-12-03 [E] [expires: 2027-12-03]

```

Далі експортую відкритий ключ в ASCII-файл

```

(kali㉿kali)-[~]
$ gpg --export -a "Sofiia" > user1.pubkey

```

Передаю ключ на VM Server

```

(kali㉿kali)-[~]
$ scp /home/kali/user1.pubkey root@192.168.74.100:/root/
root@192.168.74.100's password:
user1.pubkey

```

Перевірка на VM Server – бачимо файл

```
(root@kali)-[~]  
# ls /root/  
  
file.txt  user1.pubkey
```

Далі на VM Client створюю текстовий файл та створюю підпис (clearsign)

```
(kali@kali)-[~]  
$ echo "texttext" > testfile.txt  
  
(kali@kali)-[~]  
$ gpg --clearsign testfile.txt
```

Передаю файл із підписом на VM Server – бачу результат

```
(kali@kali)-[~]  
$ scp testfile.txt.asc root@192.168.74.100:/root/  
  
root@192.168.74.100's password:  
testfile.txt.asc
```

2.2 На VM Server імпортуйте ключ User1:

```
#gpg --import user1.pubkey
```

```
#gpg --list-public-keys
```

Перевірте ЦП отриманого файла:

```
#gpg --verify testfile.asc
```

На VM Server також згенеруйте відкритий\таємний ключ User2:

```
#gpg --gen-key
```

```
#gpg --list-public-keys
```

Тепер експортуйте відкритий ключ User2:

```
#gpg --export -a "User2" >user2.pubkey
```

та скопіюйте на VM Client:

```
#scp user2.pubkey 192.168.74.10:
```

На VM Server створіть файли test.file и test1.file та введіте в них якісь дані:

```
#touch test.file
```

Зашифруйте перший файл ключом адресата (User1):

```
#gpg -e -r 0BC2E8D9 -a test.file
```

(Тут параметр 0BC2E8D9 – key_id)

Шифрування файла з ЦП:

```
#gpg -e -s -r 0BC2E8D9 -a test1.file
```

Скопіюйте файли на VM Client:

```
#scp test.file.asc 192.168.74.10:
```

```
#scp test1.file.asc 192.168.74.10:
```

```
#gpg --import user2.pubkey
```

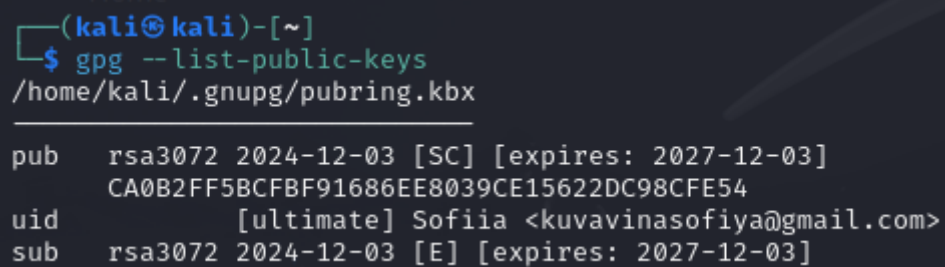
gpg: /home/kali/.gnupg/trustdb.gpg: trustdb created

gpg: key CE15622DC98CFE54: "Sofiia <kuvavinasofiya@gmail.com>" imported

gpg: Total number processed: 1

gpg: imported: 1

#gpg --list-public-keys



```
(kali㉿kali)-[~]  
$ gpg --list-public-keys  
/home/kali/.gnupg/pubring.kbx  
-----  
pub   rsa3072 2024-12-03 [SC] [expires: 2027-12-03]  
      CA0B2FF5BCFBF91686EE8039CE15622DC98CFE54  
uid           [ultimate] Sofiia <kuvavinasofiya@gmail.com>  
sub   rsa3072 2024-12-03 [E] [expires: 2027-12-03]
```

#gpg --verify testfile.asc

gpg: Signature made Mon 16 Dec 2024 11:02:32 AM EST

gpg: using RSA key C6F4CEF76A28D2EB048615582BF8A1B41B333614

gpg: Good signature from "Sofiia <kuvavinasofiya@gmail.com >" [unknown]

gpg: WARNING: This key is not certified with a trusted signature!

gpg: There is no indication that the signature belongs to the owner.

Primary key fingerprint: C6F4 CEF7 6A28 D2EB 0486 1558 2BF8 A1B4 1B78 3514

gpg --gen-key

```

(kali@kali)-[~]
$ gpg --gen-key
gpg (GnuPG) 2.2.43; Copyright (C) 2023 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Note: Use "gpg --full-generate-key" for a full featured key generation dialog.

GnuPG needs to construct a user ID to identify your key.

Real name: soffia
Email address: kuvavinasofiya@gmail.com
You selected this USER-ID:
"soffia <kuvavinasofiya@gmail.com>"

Change (N)ame, (E)mail, or (O)kay/(Q)uit? o
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
gpg: revocation certificate stored as '/home/kali/.gnupg/openpgp-revocs.d/1CE6763BA719964D3DB16A95CFB8C8FA81BA9FFE.rev'
public and secret key created and signed.

pub   rsa3072 2024-12-17 [SC] [expires: 2027-12-17]
       1CE6763BA719964D3DB16A95CFB8C8FA81BA9FFE
uid     soffia <kuvavinasofiya@gmail.com>
sub     rsa3072 2024-12-17 [E] [expires: 2027-12-17]

```

gpg --list-public-keys

```

(kali@kali)-[~]
$ gpg --list-public-keys
gpg: checking the trustdb
gpg: marginals needed: 3 completes needed: 1 trust model: pgp
gpg: depth: 0 valid: 2 signed: 0 trust: 0-, 0q, 0n, 0m, 0f, 2u
gpg: next trustdb check due at 2027-12-03
/home/kali/.gnupg/pubring.kbx

pub   rsa3072 2024-12-03 [SC] [expires: 2027-12-03]
       CA0B2FF5BCFBF91686EE8039CE15622DC98CFE54
uid     [ultimate] Sofiia <kuvavinasofiya@gmail.com>
sub     rsa3072 2024-12-03 [E] [expires: 2027-12-03]

pub   rsa3072 2024-12-17 [SC] [expires: 2027-12-17]
       1CE6763BA719964D3DB16A95CFB8C8FA81BA9FFE
uid     [ultimate] soffia <kuvavinasofiya@gmail.com>
sub     rsa3072 2024-12-17 [E] [expires: 2027-12-17]

```

#scp kali@192.168.12.21:~/server.pubkey ~/Desktop

kali@192.168.74.100's password:

server.pubkey

#date +%s >test.file

#date +%s >test1.file

#gpg -e -r FA0FF6FBD87D48D4E82B68BCAB85754A0EC36B56 -a test.file

#gpg -e -s -r FA0FF6FBD87D48D4E82B68BCAB85754A0EC36B56 -a test1.file

```
#scp kali@192.168.12.21:~/test.file.asc ~/Desktop
```

kali@192.168.12.21's password:

test.file.asc

```
#scp kali@192.168.12.21:~/test1.file.asc ~/Desktop
```

kali@192.168.12.21's password:

test1.file.asc

2.3 На VM Client імпортуйте ключ User2:

```
#gpg --import user2.pubkey
```

```
#gpg --list-public-keys
```

Перевірте геш нового ключа:

```
#gpg --fingerprint <key_id>
```

Розшифруйте файли та перевірте ЦП:

```
gpg <file>
```

Щоб видалити пару відкритий / таємний ключ, скористайтеся командою:

```
#gpg --delete-secret-and-public-key <key_id>
```

Імпорт ключа на VM Server

Імпортую ключ користувача з VM Client

```
(root@kali)-[~]
# gpg --import /root/user1.pubkey

gpg: directory '/root/.gnupg' created
gpg: keybox '/root/.gnupg/pubring.kbx' created
gpg: /root/.gnupg/trustdb.gpg: trustdb created
gpg: key 62D1D2171937222A: public key "Sofia <kuvavinasofia@gmail.com>" imported
gpg: Total number processed: 1
gpg:             imported: 1
```

Перевіряю, що ключ імпортовано

```
(root@kali)-[~]
# gpg --list-public-keys

/root/.gnupg/pubring.kbx
-----
pub   rsa3072 2024-12-03 [SC] [expires: 2027-12-03]
      CE47418DCA723B3D7FB85FA062D1D2171937222A
uid           [ unknown] Sofia <kuvavinasofia@gmail.com>
sub   rsa3072 2024-12-03 [E] [expires: 2027-12-03]
```

І також перевіряю підпис отриманого файлу

```
(root@kali)-[~]
# gpg --verify /root/testfile.txt.asc

gpg: Signature made Tue 03 Dec 2024 03:06:27 PM EST
gpg: using RSA key CE47418DCA723B3D7FB85FA062D1D2171937222A
gpg: Good signature from "Sofiia <kuvavinasofia@gmail.com>" [unknown]
gpg: WARNING: This key is not certified with a trusted signature!
gpg: There is no indication that the signature belongs to the owner.
Primary key fingerprint: CE47 418D CA72 3B3D 7FB8 5FA0 62D1 D217 1937 222A

(kali@kali)-[~]
#
```

Створюю новий ключ для VM Server

```
(kali@kali)-[~]
$ gpg --gen-key

gpg (GnuPG) 2.2.43; Copyright (C) 2023 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

gpg: keybox '/home/kali/.gnupg/pubring.kbx' created
Note: Use "gpg --full-generate-key" for a full featured key generation dialog.

GnuPG needs to construct a user ID to identify your key.

Real name: Sofiia
Email address: kuvavinasofia@gmail.com
You selected this USER-ID:
"Sofiia <kuvavinasofia@gmail.com>"

Change (N)ame, (E)mail, or (O)kay/(Q)uit? o
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
gpg: /home/kali/.gnupg/trustdb.gpg: trustdb created
gpg: directory '/home/kali/.gnupg/openpgp-revocs.d' created
gpg: revocation certificate stored as '/home/kali/.gnupg/openpgp-revocs.d/CA0B2FF5BCFBF91686EE8039CE15622DC98CFE54.rev'
public and secret key created and signed.

pub  rsa3072 2024-12-03 [SC] [expires: 2027-12-03]
    CA0B2FF5BCFBF91686EE8039CE15622DC98CFE54
uid                               Sofiia <kuvavinasofia@gmail.com>
sub  rsa3072 2024-12-03 [E] [expires: 2027-12-03]
```

Експортую новий відкритий ключ

```
(kali@kali)-[~]
$ gpg --export -a "Sofiia" > user2.pubkey
```

Передаю ключ на VM Client

```
(kali@kali)-[~]
$ scp user2.pubkey kali@192.168.74.10:/tmp/

kali@192.168.74.10's password:
user2.pubkey
```

На VM Server створюю файл для шифрування

```
(kali㉿kali)-[~]  
$ echo "cipher" > test.file
```

І шифрую файл відкритим ключем User1

У результаті буде створений файл test.file.asc

```
(kali㉿kali)-[~]  
$ gpg -e -r "Sofiia" -a test.file  
  
gpg: checking the trustdb  
gpg: marginals needed: 3 completes needed: 1 trust model: pgp  
gpg: depth: 0 valid: 1 signed: 0 trust: 0-, 0q, 0n, 0m, 0f, 1u  
gpg: next trustdb check due at 2027-12-03
```

Створюю ще один файл із шифруванням і підписом

```
(kali㉿kali)-[~]  
$ echo "cipheeer" > test1.file  
  
(kali㉿kali)-[~]  
$ gpg -e -s -r "Sofiia" -a test1.file
```

Передаю зашифровані файли на VM Client

```
(kali㉿kali)-[~]  
$ scp test.file.asc test1.file.asc kali@192.168.74.10:/tmp/  
  
kali@192.168.74.10's password:  
test.file.asc  
test1.file.asc
```

MOONCHILD

Застосування SSL

3.1 Створення і перевірка сертифіката SSL.

Створіть таємний ключ:

```
#dd if=/dev/random of=.rnd count=64
```

*Якщо у вашій системі команда “зависає” надовго, використовуйте /dev/urandom

```
#openssl genrsa -rand .rnd -out priv.key -aes256 2048
```

Створіть сертифікат-запит:

```
#openssl req -new -key priv.key -out certif.csr
```

і в інтерактивному режимі задайте параметри сертифіката.

* У параметрах сертифіката обов'язково вкажіть ваші прізвище, ім'я, групу і номер залікової книжки.

Створіть самопідписаний сертифікат:

```
#openssl x509 -req -sha256 -signkey priv.key -in certif.csr -out certif.crt -days 1830
```

* Якщо у навчальній системі відсутній алгоритм sha256, використовуйте sha1.

Зверніть увагу,

що sha1 і sha - це різні геш-функції. Для реальних додатків sha і sha1 вважаються не стійкими.

Виведіть дані сертифікату в текстовому форматі:

```
#openssl x509 -text -in certif.crt
```

та збережіть окремим файлом:

```
#openssl x509 -text -in certif.crt >certif.txt.crt
```

Експортуйте відкритий ключ:

```
openssl x509 -in certif.txt.crt -pubkey
```

```
openssl x509 -in certif.crt -pubkey >pubkey.pem
```

Перевірте сертифікат:

```
#openssl verify certif.txt.crt
```

Добавте сертифікат в список довірених:

```
#openssl x509 -text -in certif.txt.crt >> /usr/share/ssl/cert.pem
```

* У вашій версії системи сховище довірених сертифікатів може перебувати в іншому каталозі.

Знову перевірте сертифікат:

```
#openssl verify certif.txt.crt
```

Скопіюйте файл сертифіката в ОС Windows і перегляньте його там.

Створюю приватний ключ

```
(kali@kali)-[~]  
$ openssl genrsa -out priv.key 2048
```

Генерую сертифікат-запит та заповнюю дані: код країни, область, місто, назва навч. установи, факультет, піб та група, ел.пошта

```
(kali@kali)-[~]  
$ openssl req -new -key priv.key -out certif.csr  
  
You are about to be asked to enter information that will be incorporated  
into your certificate request.  
What you are about to enter is what is called a Distinguished Name or a DN.  
There are quite a few fields but you can leave some blank  
For some fields there will be a default value,  
If you enter '.', the field will be left blank.  
-----  
Country Name (2 letter code) [AU]:UA  
State or Province Name (full name) [Some-State]:Kyiv  
Locality Name (eg, city) []:Kyiv  
Organization Name (eg, company) [Internet Widgits Pty Ltd]:KPI  
Organizational Unit Name (eg, section) []:IPT  
Common Name (e.g. server FQDN or YOUR name) []:Kuvavina Sofia FB-34  
Email Address []:kuvavinasofia@gmail.com  
  
Please enter the following 'extra' attributes  
to be sent with your certificate request  
A challenge password []:  
An optional company name []:
```

Підписую сертифікат:

```
(kali㉿kali)-[~]
$ openssl x509 -req -sha256 -signkey priv.key -in certif.csr -out certif.crt -days 1830

Certificate request self-signature ok
subject=C=UA, ST=Kyiv, L=Kyiv, O=KPI, OU=IPT, CN=Kuvavina Sofiia FB-34, emailAddress=kuvavinasofia@gmail.com
```

Перевірка

Вивожу сертифікат у текстовому форматі

```
(kali㉿kali)-[~]
$ openssl x509 -text -in certif.crt

Certificate:
  Data:
    Version: 3 (0x2)
    Serial Number:
      31:81:56:55:5e:50:f7:6a:82:35:d0:4b:98:6c:86:99:95:f5:bf:b4
    Signature Algorithm: sha256WithRSAEncryption
    Issuer: C=UA, ST=Kyiv, L=Kyiv, O=KPI, OU=IPT, CN=Kuvavina Sofiia FB-34, emailAddress=kuvavinasofia@gmail.com
    Validity
      Not Before: Dec  4 11:23:38 2024 GMT
      Not After : Dec  8 11:23:38 2029 GMT
    Subject: C=UA, ST=Kyiv, L=Kyiv, O=KPI, OU=IPT, CN=Kuvavina Sofiia FB-34, emailAddress=kuvavinasofia@gmail.com
    Subject Public Key Info:
      Public Key Algorithm: rsaEncryption
      Public-Key: (2048 bit)
      Modulus:
        00:9d:71:52:ad:0d:7f:5a:c0:ff:1e:29:68:78:ea:
        f2:a2:61:a3:12:4f:3b:62:c5:9c:64:ad:52:70:4a:
        cd:ba:67:d9:32:bd:03:2d:b3:0c:97:18:c3:cc:84:
        0d:1f:b4:12:07:30:0d:a9:33:ad:3a:f7:01:e2:33:
        58:e3:3b:8c:bc:8e:37:96:b3:69:d7:b3:bb:16:70:
        62:1b:e8:60:33:33:e3:6b:d5:94:be:5a:a4:bb:43:
        e4:c7:a9:95:e0:aa:e5:57:2e:bd:59:d5:f3:ae:05:
        3f:11:27:23:84:8a:3f:1c:d8:9c:ce:3b:78:41:76:
        b2:c1:c1:52:70:fc:44:d5:57:b1:c9:88:b8:52:57:
        7f:a5:8c:1c:ed:65:6f:fe:36:0a:1e:79:8c:d4:ab:
        34:7b:b6:f0:1f:c1:49:9f:60:ab:b3:11:d1:a8:62:
        2d:61:10:a4:57:cd:8f:f2:e9:69:73:f3:08:37:1c:
        26:22:a3:43:c7:56:14:c2:17:00:f7:2c:49:80:1a:
        2f:c6:43:ba:85:56:2b:fd:bf:fd:b6:db:91:67:fc:
        09:6e:17:4c:77:bc:2f:0c:6d:da:10:82:1e:61:99:
        46:02:1f:6e:a0:80:aa:e1:fc:9b:61:06:f3:23:30:
        1e:f0:3a:2d:67:ff:cd:4b:a0:42:b7:28:56:0b:4e:
        96:69
      Exponent: 65537 (0x10001)
    X509v3 extensions:
      X509v3 Subject Key Identifier:
        07:AB:F8:AF:5D:09:96:D5:E8:8B:6B:89:8E:D4:F5:0D:85:42:5C:57
    Signature Algorithm: sha256WithRSAEncryption
    Signature Value:
      81:ca:87:90:28:04:b1:4a:0f:ca:85:6a:db:08:df:7a:c3:62:
      eb:52:b7:bf:08:4e:e1:45:05:6b:0f:27:39:cd:87:24:97:bb:
      ec:bf:5e:1c:2c:38:21:fe:0f:ae:5d:c1:42:77:b6:93:db:86:
      da:7f:a8:82:5e:41:2e:c2:c4:87:4b:e1:ca:72:19:1b:40:4a:
      bb:16:7e:a8:00:5a:01:59:95:31:41:68:2c:c2:89:50:29:c4:
```

Зберігаю сертифікат у текстовому файлі та еспортую публічний ключ

```
(kali㉿kali)-[~]
$ openssl x509 -text -in certif.crt > certiftxt.crt

(kali㉿kali)-[~]
$ openssl x509 -in certif.crt -pubkey > pubkey.pem
```

Перевіряю сертифікат (він самопідписаний, тому OpenSSL не може його перевірити проти списку довірених сертифікатів)

```
(kali@kali)-[~]
$ openssl verify certiftxt.crt

C=UA, ST=Kyiv, L=Kyiv, O=KPI, OU=IPT, CN=Kuvavina Sofia FB-34, emailAddress=kuvavinasofia@gmail.com
error 18 at 0 depth lookup: self-signed certificate
error certiftxt.crt: verification failed
```

копіюю сертифікат у каталог сертифікатів, перегенерую сертифікати і знову перевіряю бачу, що сертифікат довірений

```
(kali@kali)-[~]
$ sudo cp certiftxt.crt /etc/ssl/certs/

(kali@kali)-[~]
$ sudo c_rehash /etc/ssl/certs/

Doing /etc/ssl/certs/
WARNING: Skipping duplicate certificate ca-certificates.crt
WARNING: Skipping duplicate certificate ca-certificates.crt

(kali@kali)-[~]
$ openssl verify certiftxt.crt

certiftxt.crt: OK
```

копіюю сертифікат на Windows для перегляду(перед цим на Windows-машині виконую команду ipconfig)

```
(kali@kali)-[~]
$ scp certiftxt.crt user@192.168.100.2:/path/to/destination

ssh: connect to host 192.168.100.2 port 22: No route to host
scp: Connection closed
```

3.2 Шифрування файлів.

Згенеруйте ключ для симетричного шифрування файлу:

```
#openssl rand -out key.bin 64
```

Зашифруйте деякий достатньо великий файл:

```
#openssl enc -aes-256-cbc -salt -in largfile.dat -out largfile.dat.enc -pass file:./key.bin
```

Зашифруйте ключ шифрування файла асиметричним алгоритмом

(використовуйте відкритий ключ):

```
#openssl rsautl -encrypt -inkey pubkey.pem -pubin -in key.bin -out key.bin.enc
```

або використовуйте відповідний сертифікат:

```
#openssl rsautl -encrypt -inkey certif.crt -certin -in key.bin -out key.bin.enc2
```

Переконайтеся в ідентичності двох отриманих файлів.

Розшифруйте ключ шифрування:

```
#openssl rsautl -decrypt -inkey priv.key -in key.bin.enc -out key.bin2
```

Потім розшифруйте файл:

```
#openssl enc -d -aes-256-cbc -in largfile.dat.enc -out largfile.dat2 -pass file:./key.bin2
```

Порівняйте вихідний та розшифрований файли.

Створюю симетричний ключ

```
(kali㉿kali)-[~]  
$ openssl rand -out key.bin 64
```

Зашифрую файл

```
(kali㉿kali)-[~]  
$ openssl enc -aes-256-cbc -pbkdf2 -iter 100000 -salt -in largfile.dat -out largfile.dat.enc -pass file:./key.bin
```

Шифрування ключа асиметричним алгоритмом

Використовую сертифікат та публічний ключ для шифрування

```
(kali㉿kali)-[~]  
$ openssl pkeyutl -encrypt -inkey certif.crt -certin -in key.bin -out key.bin.enc2
```

```
(kali㉿kali)-[~]  
$ openssl pkeyutl -encrypt -inkey pubkey.pem -pubin -in key.bin -out key.bin.enc
```

Перевіряю ідентичність файлів

```
(kali㉿kali)-[~]  
$ diff key.bin.enc key.bin.enc2  
  
Binary files key.bin.enc and key.bin.enc2 differ
```

Далі розшифрую ключ

```
(kali㉿kali)-[~]  
$ openssl pkeyutl -decrypt -inkey priv.key -in key.bin.enc -out key.bin2
```

розшифрую файл

```
(kali㉿kali)-[~]  
$ openssl enc -d -aes-256-cbc -pbkdf2 -in largfile.dat.enc -out largfile.dat2 -pass file:./key.bin
```

Перевіряю ідентичність файлів

```
(kali㉿kali)-[~]  
$ diff largfile.dat largfile.dat2
```

THE COURT OF THE CRIMSON KING

Цифровий підпис

Створіть деякий документ для тестування механізму цифрового підпису:

```
#echo 'data to sign' > example.txt
```

*В тексті вкажіть свою групу і номер своєї залікової книжки.

Розрахуйте значення цифрового підпису тестового документа (значення геш-функції, що зашифроване таємним ключем):

```
#openssl dgst -sha256 -sign priv.key -out example.sha example.txt
```

Використовуючи відкритий ключ, перевірте коректність цифрового підпису:

```
#openssl dgst -sha256 -verify pubkey.pem -signature example.sha example.txt
```

Інший варіант обчислення цифрового підпису:

Збережіть значення геш-функції документа окремим файлом:

```
#openssl dgst -sha256 example.txt > example.hash
```

Зашифруйте відкритим ключем отриманий файл:

```
#openssl rsautl -sign -inkey priv.key -keyform PEM -in example.hash
```

```
>example.hash.sign
```

Використовуючи відкритий ключ, проведіть зворотнє перетворення:

```
#openssl rsautl -verify -pubin -inkey pubkey.pem -keyform PEM -in
```

```
example.hash.sign
```

Порівняйте отримане на попередньому кроці значення геш-функції зі значенням геш-функції,

обчисленої на основі вмісту файлів:

```
#openssl dgst -sha256 example.txt
```

У каталозі, де знаходяться файли, створені в роботі, виконайте команду:

```
#ls -l
```

Створюю файл

```
(kali㉿kali)-[~]  
$ echo 'FB-34 IPT' > example.txt
```

Розраховую цифровий підпис і перевіряю його – все добре

```
(kali㉿kali)-[~]  
$ openssl dgst -sha256 -sign priv.key -out example.sha example.txt  
  
(kali㉿kali)-[~]  
$ openssl dgst -sha256 -verify pubkey.pem -signature example.sha example.txt  
Verified OK
```

Інший спосіб

Зберігаю геш функцію

```
(kali㉿kali)-[~]  
$ openssl dgst -sha256 example.txt > example.hash
```

Обчислюю геш, підписую його приватним ключем і перевіряю підпис відкритим ключем
– бачу позитвний результат

```
(kali㉿kali)-[~]  
$ openssl dgst -sha256 -binary -out example.hash example.txt  
  
(kali㉿kali)-[~]  
$ openssl pkeyutl -sign -inkey priv.key -keyform PEM -in example.hash -out example.hash.sign  
  
(kali㉿kali)-[~]  
$ openssl pkeyutl -verify -pubin -inkey pubkey.pem -in example.hash -sigfile example.hash.sign  
Signature Verified Successfully
```

Порівнюю геші

```
(kali㉿kali)-[~]  
$ openssl dgst -sha256 example.txt  
  
SHA2-256(example.txt)= 944aee0848d1a4d077153994d1aa9f459dac9dcbbab1d4fbaff5905ad4d30d7f
```

Переглядаю всі файли в каталозі

(kali㉿kali)-[~]

\$ ls -l

Home

total 112

-rw-rw-r--	1	kali	kali	1383	Dec	4	06:23	certif.crt
-rw-rw-r--	1	kali	kali	1058	Dec	4	06:21	certif.csr
-rw-rw-r--	1	kali	kali	4383	Dec	4	06:24	certif.txt.crt
drwxr-xr-x	2	kali	kali	4096	Dec	3	13:51	Desktop
drwxr-xr-x	2	kali	kali	4096	Dec	3	14:32	Documents
drwxr-xr-x	2	kali	kali	4096	Dec	3	13:51	Downloads
-rw-rw-r--	1	kali	kali	32	Dec	4	07:21	example.hash
-rw-rw-r--	1	kali	kali	256	Dec	4	07:21	example.hash.sign
-rw-rw-r--	1	kali	kali	256	Dec	4	07:15	example.sha
-rw-rw-r--	1	kali	kali	10	Dec	4	07:15	example.txt
-rw-rw-r--	1	kali	kali	64	Dec	4	06:46	key.bin
-rw-rw-r--	1	kali	kali	64	Dec	4	06:56	key.bin2
-rw-rw-r--	1	kali	kali	256	Dec	4	06:55	key.bin.enc
-rw-rw-r--	1	kali	kali	256	Dec	4	06:53	key.bin.enc2
-rw-rw-r--	1	kali	kali	5	Dec	4	06:49	largfile.dat
-rw-rw-r--	1	kali	kali	5	Dec	4	07:00	largfile.dat2
-rw-rw-r--	1	kali	kali	32	Dec	4	06:59	largfile.dat.enc
drwxr-xr-x	2	kali	kali	4096	Dec	3	13:51	Music
drwxr-xr-x	2	kali	kali	4096	Dec	3	15:00	Pictures
-rw-rw-r--	1	kali	kali	1704	Dec	4	06:19	priv.key
-rw-rw-r--	1	kali	kali	1834	Dec	4	06:25	pubkey.pem
drwxr-xr-x	2	kali	kali	4096	Dec	3	13:51	Public
drwxr-xr-x	2	kali	kali	4096	Dec	3	13:51	Templates
-rw-rw-r--	1	kali	kali	9	Dec	3	15:06	testfile.txt
-rw-rw-r--	1	kali	kali	717	Dec	3	15:06	testfile.txt.asc
-rw-rw-r--	1	kali	kali	2452	Dec	3	14:55	user1.pubkey
drwxr-xr-x	2	kali	kali	4096	Dec	3	13:51	Videos